



Instituto Politécnico Nacional
Escuela Superior de Ingeniería Mecánica y Eléctrica
Unidad Culhuacan



Maestría en Ingeniería en Seguridad y Tecnologías de la Información

Tarea SVM

Profesora:

Mariko Nakano

Alumno:

Leonardo Vicente Jiménez

Noviembre 2025

Indice

INTRODUCCIÓN.....	3
OBJETIVOS ESPECIFICOS.....	3
DESARROLLO.....	3
Configuración de entorno de trabajo.....	3
01. Usar todos los datos (4 características).....	4
02. Obtener resultados variando el número de vecinos y usando varias métricas para evaluar los resultados.....	5
CONCLUSIÓN.....	7

INTRODUCCIÓN

Las Máquinas de Vectores de Soporte (SVM) constituyen uno de los algoritmos más robustos y versátiles en el ámbito del aprendizaje automático, destacando particularmente en problemas de clasificación. Su eficacia radica en la capacidad de encontrar hiperplanos óptimos de separación entre clases, incluso en espacios de alta dimensionalidad. El presente trabajo tiene como objetivo principal implementar y evaluar el desempeño de clasificadores SVM aplicados al dataset de cáncer de mama, explorando diferentes configuraciones de kernels y parámetros. La relevancia de este estudio reside en la comparación sistemática de kernels lineales, RBF y polinómicos, analizando su capacidad para discernir entre muestras benignas y malignas. Mediante técnicas de validación cruzada y optimización de hiperparámetros, se busca identificar la configuración óptima que maximice la métrica F1-score, considerada ideal para problemas con posibles desbalances entre clases.

El desarrollo de esta investigación sigue una metodología estructurada que incluye: preparación del entorno computacional, análisis exploratorio de datos, implementación de pipelines de machine learning, y evaluación comparativa de modelos. Todo ello con el propósito de establecer mejores prácticas en la aplicación de SVM para diagnóstico médico asistido.

OBJETIVOS ESPECIFICOS

- Realizar la clasificación usando más de dos características
- Probar diferentes kernels y diferentes grados en kernel de polinomio
- Realizar análisis de los resultados obtenidos.

DESARROLLO

Configuración de entorno de trabajo

Para el desarrollo de estas tareas se requirió el uso de los siguientes paquetes, los cuales serán listados con su respectiva versión utilizada

- **numpy** **2.2.6**
- **pandas** **2.3.2**
- **scikit-learn** **1.7.1**
- **seaborn** **0.13.2**
- **Matplotlib** **3.10.3**

Los paquetes mencionados anteriormente fueron utilizados dentro de un entorno virtual con el fin de evitar conflictos con versiones instaladas en el sistema base; dicho entorno virtual fue creado mediante el siguiente comando

```
$ python3 -m venv claseReconocimiento
```

Y la instalación de cada paquete dentro de este entorno virtual fue realizado mediante

\$ pip install [nombre del paquete a instalar]

Además se menciona que para la elaboración de estas tareas se empleo un solo tipo de formato de archivo, siendo este el realizado mediante cuadernos de ipynb para su mejor visualización, dicho archivo se encuentra en:

<https://github.com/Kinlo-sama/reconocimientoPatrones/blob/main/Homeworks/SVM/taraeSVM.ipynb>

El primer paso para llevar a cabo el desarrollo de los puntos mencionados en los objetivos especificos es necesario la importación de paquetes a utilizar, estos pueden visualizarse en la implementación del **Codigo 01**

Codigo 1 Librerias necesarias para el desarrollo de los objetivos especificos

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
import seaborn as sns
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score
from sklearn.metrics import make_scorer, accuracy_score, precision_score, recall_score, f1_score
import warnings
warnings.filterwarnings('ignore')
```

01. Realizar la clasificación usando más de dos características

Para cumplir con el primer objetivo, es necesario cargar el conjunto de datos correctamente. Para ello, se siguieron los pasos descritos en el Código 02, el cual muestra cómo se cargan las características del dataset **breast_cancer**, de las cuales solo se tomaran las dos primeras . En este conjunto, las características de cada muestra se almacenan en las columnas de `feature_names`, mientras que las etiquetas se encuentran en un arreglo dentro del objeto `cancer`, bajo el nombre `target`, además creamos una columna para identificar de forma numerica la clase a la que pertenece cada muestra siendo esta una clasificación binaria. La visualización de las primeras muestras de los datos a utilizar se presenta en la Figura 01.

Codigo 02 Carga y visualización del dataset **breast_cancer**

```

cancer = load_breast_cancer()
feature_list=list(cancer.feature_names)
df=pd.DataFrame(data=cancer.data, columns=cancer.feature_names)
df["tipo de cancer"]=cancer.target
sub_df=df.iloc[:,2:4].copy()
sub_df["tipo de cancer"]=cancer.target
# mean perimeter - mean area - tipo de cancer
name_list =[]
for i in range(len(df)):
    tipo = 'maligno'
    if sub_df.iloc[i,2] == 1:
        tipo ='benigno'
    name_list.append(tipo)
sub_df["tipo de cancer letra"]=name_list
sub_df.head()

```

	mean perimeter	mean area	tipo de cancer
0	122.80	1001.0	0
1	132.90	1326.0	0
2	130.00	1203.0	0
3	77.58	386.1	0
4	135.10	1297.0	0

Figura 01 Visualización del dataset **breast_cancer**

Además de cargar el dataset, se realizó una gráfica que representa la relación entre cada una de las variables utilizadas en el modelo. Esta gráfica puede visualizarse en la Figura 02 y su implementación en el Código 03. Esta grafica nos dio una vista sobre el comportamiento de los datos.

Codigo 03 Implementación grafica de relación entre variables

```

sub_df["tipo de cancer letra"]=name_list
sns.set(font_scale=1.3)
sns.pairplot(sub_df.drop(columns='tipo de cancer'), hue="tipo de cancer letra",
             hue_order=['benigno','maligno'])
plt.show()

```

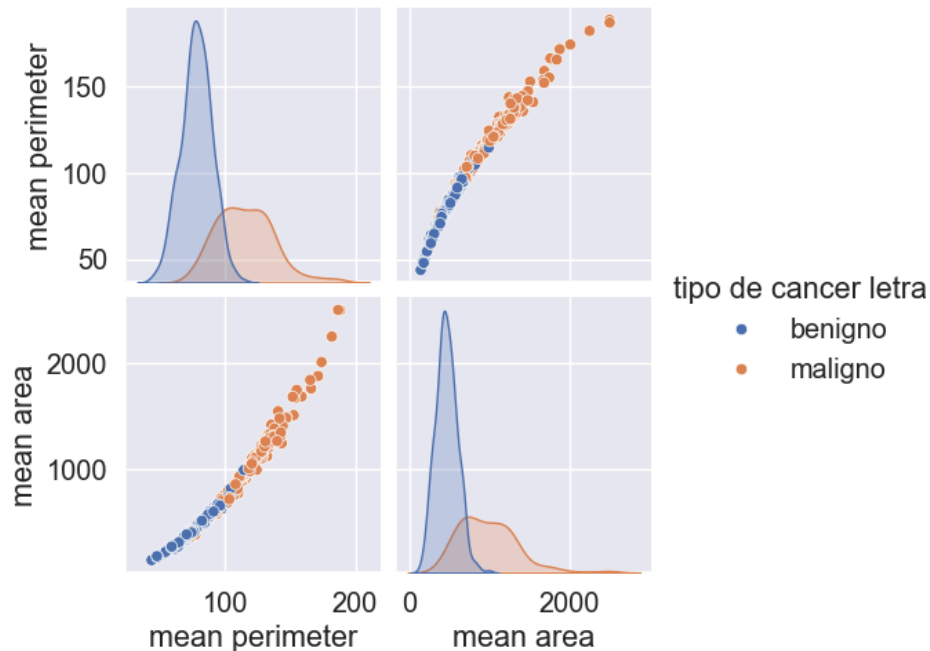


Figura 02 Grafica mostrando relación entre variables

02. Probar diferentes kernels y diferentes grados en kernel de polinomio

Para evaluar el modelo con diferentes kernels y grados polinomiales, se utilizará la implementación del Código 04. Primero, se probarán los kernels RBF y Lineal, obteniendo los mejores resultados entre estos, los cuales se visualizan en la Figura 03 (a). Posteriormente, en el Código 05, se realizará una búsqueda variando el grado del kernel polinómico. Tanto el Código 04 como el 05 optimizan los parámetros con respecto a la métrica F1-score, debido a que esta ofrece un mejor balance entre precisión y recall. Los resultado de la implementación 05 se encuentra en la Figura 03 (b).

Codigo 04 Busqueda de los mejores parametros para kernels RBF y Linear

```
grid = {
    'classifier__C': [0.1, 1, 10, 100],
    'classifier__kernel': ['rbf', 'linear'],
    'classifier__gamma': ['scale', 'auto', 0.1, 1]
}
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', SVC(random_state=42))
])
scoring = {
```

```

'accuracy': make_scorer(accuracy_score),
'precision_weighted': make_scorer(precision_score, average='weighted'),
'recall_weighted': make_scorer(recall_score, average='weighted'),
'f1_weighted': make_scorer(f1_score, average='weighted')
}
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=grid,
    cv=10,
    scoring=scoring,
    n_jobs=-1,
    verbose=0,
    refit='f1_weighted'
)
grid_search.fit(X_train, y_train)
y_pred_test = grid_search.predict(X_test)
val_accuracy = accuracy_score(y_test, y_pred_test)
for param, value in grid_search.best_params_.items():
    print(f" {param}: {value}")
print(f"Mejor score (CV): {grid_search.best_score_:.4f}")
print(f"Test: {val_accuracy:.4f}")
print()

```

Codigo 05 Busqueda de mejores parametros para el kernel polinomico

```

poly_grid = {
    'classifier__C': [0.1, 1, 10],
    'classifier__gamma': ['scale', 0.1, 1],
    'classifier__degree': [2, 3, 4, 5, 6],
    'classifier__coef0': [0.0, 1.0, 1.5]
}
poly_search = GridSearchCV(
    estimator=pipeline,
    param_grid=poly_grid,
    cv=3, # Solo 3 folds
    scoring='f1_weighted',
    n_jobs=-1,
    verbose=0,
    refit=True
)
poly_search.fit(X_train, y_train)
y_pred_test_poly = poly_search.predict(X_test)
val_accuracy_poly = accuracy_score(y_test, y_pred_test)
for param, value in poly_search.best_params_.items():
    print(f" {param}: {value}")
print(f"Mejor score (CV): {poly_search.best_score_:.4f}")
print(f"Test: {val_accuracy_poly:.4f}")
print()

```

<pre> classifier__C: 10 classifier__gamma: scale classifier__kernel: linear Mejor score (CV): 0.8903 Test: 0.8671 </pre>	<pre> classifier__C: 0.1 classifier__coef0: 0.0 classifier__degree: 2 classifier__gamma: 1 Mejor score (CV): 0.8903 Test: 0.8671 </pre>
--	---

(a)

(b)

Figura 03 Resultados de los mejores parámetros para (a) kernels lineal y RBF, y para (b) kernel polinómico

Con el fin de automatizar la selección del mejor modelo entre los kernels lineal, RBF y polinómicos, se implementa el siguiente código después de completar las búsquedas de mejores parámetros realizadas anteriormente.

```
best_estimator = poly_search if val_accuracy_poly > val_accuracy else grid_search
```

03. Realizar análisis de los resultados obtenidos.

El análisis se realizará mediante la evaluación gráfica del mejor modelo obtenido, tal como se muestra en la implementación del Código 06. Esta genera una gráfica que ilustra cómo el mejor modelo realiza la separación de los datos, la cual se visualiza en la Figura 04.

Código 06 Implementación para graficar un contorno que divida los datos mediante el mejor modelo

```

X1, X2 = np.meshgrid(np.arange(start = X_set[:, 0].min() - 1, stop = X_set[:, 0].max() + 1,
                             step = 0.1),
                    np.arange(start = X_set[:, 1].min() - 1, stop = X_set[:, 1].max() + 1,
                             step = 0.1))
plt.contourf(X1, X2, best_estimator.predict(np.array([X1.ravel(),
X2.ravel()]).T).reshape(X1.shape),
             alpha = 0.4, cmap = ListedColormap(('red', 'green')))
plt.xlim(X1.min(), X1.max())
plt.ylim(X2.min(), X2.max())
for i, j in enumerate(np.unique(y)):
    plt.scatter(X[y == j, 0], X[y == j, 1], s=10,
               color = ListedColormap(('red', 'green'))(i), label = j)
plt.title('Test Data: Accuracy='+str(np.round(score,2)))
plt.legend()
plt.show()

```

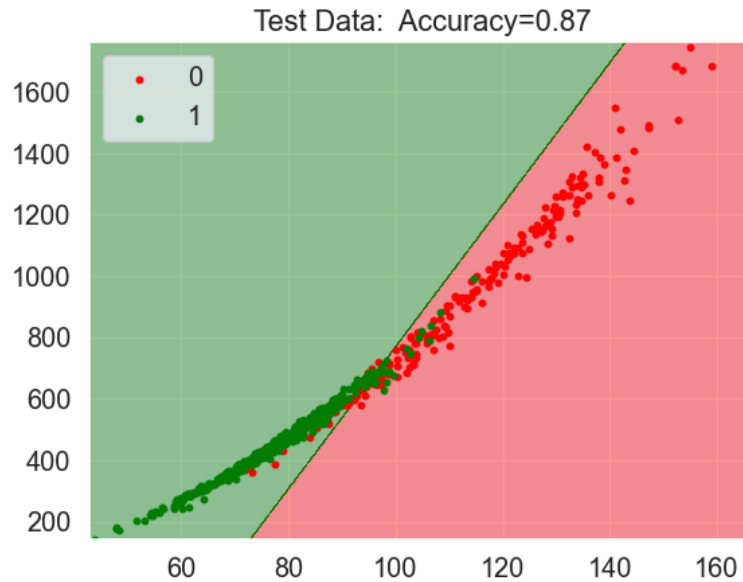



Figura 04 Separación de los datos mediante el mejor modelo (lineal)

Además de mostrar el mejor modelo, que como se observa en la Figura 04 corresponde a un kernel lineal, también se presenta el modelo polinómico dado que ambos kernels obtuvieron resultados idénticos. La visualización de la separación de datos mediante el modelo polinómico se muestra en la Figura 05.

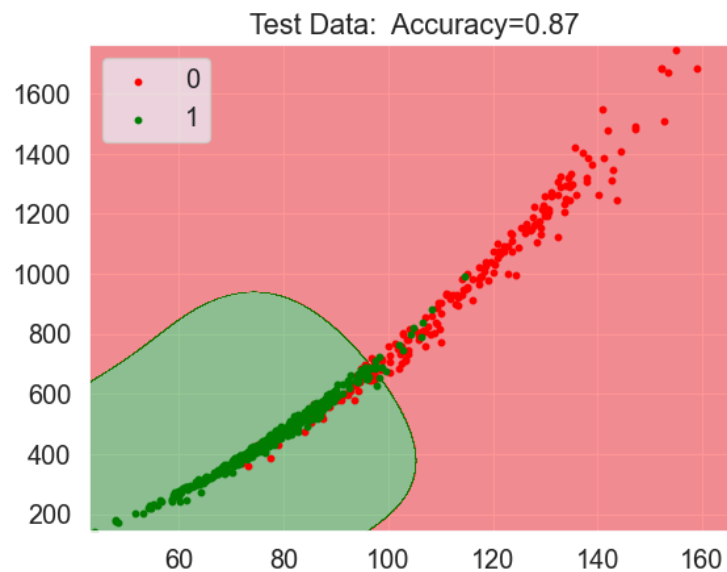


Figura 05 Separación de los datos mediante el mejor modelo (polinómico)

CONCLUSIÓN

El análisis comparativo de kernels SVM realizado sobre el dataset de cáncer de mama revela hallazgos significativos en cuanto al desempeño de clasificación. Los resultados demuestran que tanto el kernel lineal como el polinómico de grado 2 alcanzan un rendimiento idéntico en

términos de métricas de evaluación, específicamente en F1-score, precisión y exactitud. Esta equivalencia en performance sugiere que la relación de separabilidad entre las clases benignas y malignas puede ser capturada efectivamente por un modelo lineal, indicando una posible naturaleza linealmente separable del problema con las características seleccionadas.